PLSCanonical

class sklearn.cross_decomposition.PLSCanonical(*n_components=2, *, scale=True, algorithm='nipals', max_iter=500, tol=1e-06, copy=True*) [source]

Partial Least Squares transformer and regressor.

For a comparison between other cross decomposition algorithms, see Compare cross decomposition methods.

Read more in the User Guide.

> *Added in version 0.8.*

Parameters:

n_components : *int, default=2*

Number of components to keep. Should be in `[1, min(n_samples, n_features, n_targets)]`.

scale : *bool, default=True*

Whether to scale `X` and `y`.

algorithm : *{'nipals', 'svd'}, default='nipals'*

The algorithm used to estimate the first singular vectors of the cross-covariance matrix. 'nipals' uses the power method while 'svd' will compute the whole SVD.

max_iter : *int, default=500*

The maximum number of iterations of the power method when `algorithm='nipals'`. Ignored otherwise.

tol : *float, default=1e-06*

The tolerance used as convergence criteria in the power method: the algorithm stops whenever the squared norm of `u_i - u_{i-1}` is less than `tol`, where `u` corresponds to the left singular vector.

copy : *bool, default=True*

Whether to copy `X` and `y` in fit before applying centering, and potentially scaling. If False, these operations will be done inplace, modifying both arrays.

Attributes:

x_weights_ : *ndarray of shape (n_features, n_components)*

The left singular vectors of the cross-covariance matrices of each iteration.

y_weights_ : *ndarray of shape (n_targets, n_components)*

The right singular vectors of the cross-covariance matrices of each iteration.

x_loadings_ : *ndarray of shape (n_features, n_components)*

The loadings of `X`.

y_loadings_ : *ndarray of shape (n_targets, n_components)*

The loadings of `y`.

x_rotations_ : *ndarray of shape (n_features, n_components)*

The projection matrix used to transform `X`.

y_rotations_ : *ndarray of shape (n_targets, n_components)*

The projection matrix used to transform `y`.

coef_ : *ndarray of shape (n_targets, n_features)*

The coefficients of the linear model such that `y` is approximated as `y = X @ coef_.T + intercept_`.

intercept_ : *ndarray of shape (n_targets,)*

The intercepts of the linear model such that `y` is approximated as `y = X @ coef_.T + intercept_`.

> *Added in version 1.1.*

n_iter_ : *list of shape (n_components,)*

Number of iterations of the power method, for each component. Empty if `algorithm='svd'`.

n_features_in_ : *int*

Number of features seen during fit.

feature_names_in_ : *ndarray of shape (`n_features_in_` ,)*

Names of features seen during fit. Defined only when `X` has feature names that are all strings.

> *Added in version 1.0.*

See also

CCA

Canonical Correlation Analysis.

PLSSVD

Partial Least Square SVD.

Examples

```
>>> from sklearn.cross_decomposition import PLSCanonical
>>> X = [[0., 0., 1.], [1.,0.,0.], [2.,2.,2.], [2.,5.,4.]]
>>> y = [[0.1, -0.2], [0.9, 1.1], [6.2, 5.9], [11.9, 12.3]]
>>> plsca = PLSCanonical(n_components=2)
>>> plsca.fit(X, y)
PLSCanonical()
>>> X_c, y_c = plsca.transform(X, y)
```

fit(*X*, *y*) [source]

Fit model to data.

Parameters:

X : *array-like of shape (n_samples, n_features)*

Training vectors, where `n_samples` is the number of samples and `n_features` is the number of predictors.

y : *array-like of shape (n_samples,) or (n_samples, n_targets)*

Target vectors, where `n_samples` is the number of samples and `n_targets` is the number of response variables.

Returns:

self : *object*

Fitted model.

fit_transform(*X*, *y=None*) [source]

Learn and apply the dimension reduction on the train data.

Parameters:

X : *array-like of shape (n_samples, n_features)*

Training vectors, where `n_samples` is the number of samples and `n_features` is the number of predictors.

y : *array-like of shape (n_samples, n_targets), default=None*

Target vectors, where `n_samples` is the number of samples and `n_targets` is the number of response variables.

Returns:

self : *ndarray of shape (n_samples, n_components)*

Return `x_scores` if `y` is not given, `(x_scores, y_scores)` otherwise.

`get_feature_names_out(input_features=None)` [source]

Get output feature names for transformation.

The feature names out will prefixed by the lowercased class name. For example, if the transformer outputs 3 features, then the feature names out are: `["class_name0", "class_name1", "class_name2"]`.

Parameters:

input_features : *array-like of str or None, default=None*

Only used to validate feature names with the names seen in `fit`.

Returns:

feature_names_out : *ndarray of str objects*

Transformed feature names.

`get_metadata_routing()` [source]

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

Returns:

routing : *MetadataRequest*

A `MetadataRequest` encapsulating routing information.

`get_params(deep=True)` [source]

Get parameters for this estimator.

Parameters:

deep : *bool, default=True*

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Returns:

params : *dict*

Parameter names mapped to their values.

`inverse_transform(X, y=None)` [source]

Transform data back to its original space.

Parameters:

X : *array-like of shape (n_samples, n_components)*

New data, where `n_samples` is the number of samples and `n_components` is the number of pls components.

y : *array-like of shape (n_samples,) or (n_samples, n_components)*

New target, where `n_samples` is the number of samples and `n_components` is the number of pls components.

Returns:

X_original : *ndarray of shape (n_samples, n_features)*

Return the reconstructed `X` data.

y_original : *ndarray of shape (n_samples, n_targets)*

Return the reconstructed `X` target. Only returned when `y` is given.

Notes

This transformation will only be exact if `n_components=n_features`.

`predict(X, copy=True)` [source]

Predict targets of given samples.

Parameters:

X : *array-like of shape (n_samples, n_features)*

Samples.

copy : *bool, default=True*

Whether to copy `X` or perform in-place normalization.

Returns:

y_pred : *ndarray of shape (n_samples,) or (n_samples, n_targets)*

Returns predicted values.

Notes

This call requires the estimation of a matrix of shape `(n_features, n_targets)`, which may be an issue in high dimensional space.

score(*X*, *y*, *sample_weight=None*) [source]

Return coefficient of determination on test data.

The coefficient of determination, R^2, is defined as $(1 - \frac{u}{v})$, where u is the residual sum of squares `((y_true - y_pred)** 2).sum()` and v is the total sum of squares `((y_true - y_true.mean()) ** 2).sum()`. The best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts the expected value of `y`, disregarding the input features, would get a R^2 score of 0.0.

Parameters:

X : *array-like of shape (n_samples, n_features)*

Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape `(n_samples, n_samples_fitted)`, where `n_samples_fitted` is the number of samples used in the fitting for the estimator.

y : *array-like of shape (n_samples,) or (n_samples, n_outputs)*

True values for `X`.

sample_weight : *array-like of shape (n_samples,), default=None*

Sample weights.

Returns:

score : *float*

R^2 of `self.predict(X)` w.r.t. `y`.

Notes

The R^2 score used when calling `score` on a regressor uses `multioutput='uniform_average'` from version 0.23 to keep consistent with default value of `r2_score`. This influences the `score` method of all the multioutput regressors (except for `MultiOutputRegressor`).

set_output(*, *transform=None*) [source]

Set output container.

Refer to the user guide for more details and Introducing the set_output API for an example on how to use the API.

Parameters:

transform : *{"default", "pandas", "polars"}, default=None*

Configure output of `transform` and `fit_transform`.

- `"default"`: Default output format of a transformer
- `"pandas"`: DataFrame output
- `"polars"`: Polars output
- `None`: Transform configuration is unchanged

> *Added in version 1.4:* `"polars"` option was added.

Returns:

self : *estimator instance*

Estimator instance.

`set_params(**params)` [source]

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as `Pipeline`). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Parameters:

****params** : *dict*

Estimator parameters.

Returns:

self : *estimator instance*

Estimator instance.

`set_predict_request(*, copy: bool | None | str = '$UNCHANGED$')` → PLSCanonical [source]

Configure whether metadata should be requested to be passed to the `predict` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True`

(see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True`: metadata is requested, and passed to `predict` if provided. The request is ignored if metadata is not provided.
- `False`: metadata is not requested and the meta-estimator will not pass it to `predict`.
- `None`: metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str`: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

> ***Added in version 1.3.***

Parameters:

copy : *str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*

Metadata routing for `copy` parameter in `predict`.

Returns:

self : *object*

The updated object.

`set_score_request(*, sample_weight: bool | None | str = '$UNCHANGED$') → PLSCanonical` [source]

Configure whether metadata should be requested to be passed to the `score` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True`: metadata is requested, and passed to `score` if provided. The request is ignored if metadata is not provided.
- `False`: metadata is not requested and the meta-estimator will not pass it to `score`.

- `None` : metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str` : metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

> *Added in version 1.3.*

Parameters:

sample_weight : *str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*

Metadata routing for `sample_weight` parameter in `score` .

Returns:

self : *object*

The updated object.

set_transform_request(*, copy: bool | None | str = '$UNCHANGED$') → PLSCanonical [source]

Configure whether metadata should be requested to be passed to the `transform` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see **`sklearn.set_config`**). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

- `True` : metadata is requested, and passed to `transform` if provided. The request is ignored if metadata is not provided.
- `False` : metadata is not requested and the meta-estimator will not pass it to `transform` .
- `None` : metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str` : metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Parameters:

copy : *str, True, False, or None, default=sklearn.utils.metadata_routing.UNCHANGED*

Metadata routing for `copy` parameter in `transform`.

Returns:

self : *object*

The updated object.

`transform(X, y=None, copy=True)` [source]

Apply the dimension reduction.

Parameters:

X : *array-like of shape (n_samples, n_features)*

Samples to transform.

y : *array-like of shape (n_samples, n_targets), default=None*

Target vectors.

copy : *bool, default=True*

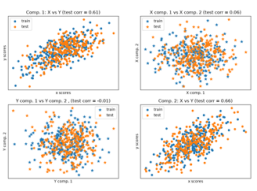
Whether to copy `X` and `y`, or perform in-place normalization.

Returns:

x_scores, y_scores : *array-like or tuple of array-like*

Return `x_scores` if `y` is not given, `(x_scores, y_scores)` otherwise.

Gallery examples



CCA

Compare cross

Next
PLSRegression ›

© Copyright 2007 - 2026, scikit-learn developers (BSD License).